**Sonia Lerner**
**CSCI3212 HW1**
**20 October 2026**

## 1.1

Let $z$ be the lowest node that becomes unbalanced (BF of 2 or $-2$) after an insertion. Say the new key went into `z.left`, and let $y =$ `z.left`. Before the insertion, $z$ was balanced and its left side was the taller one, so `z.left.height` was $h+1$, `z.right.height` was $h$, and `z.height` was $h+2$. After the insertion, `y.height` is $h+2$. Because $y$'s height grew and $z$ is the lowest unbalanced node, $y$'s BF is exactly 1 or $-1$, so $y$'s children have heights $h+1$ and $h$.

**LL case** (`y.left.height` is $h+1$, `y.right.height` is $h$): single right rotation at $z$. Now $z$ has children `y.right` (height $h$) and `z.right` (height $h$), so `z.height` is $h+1$. The new root $y$ has children of height $h+1$ and $h+1$, so `y.height` is $h+2$.

**LR case** (`y.left.height` is $h$, and $w =$ `y.right` has height $h+1$): left rotation at $y$, then right rotation at $z$. The children of $w$ have heights $h$ or $h-1$. The new root $w$ has left child $y$ (children of height $h$ and at most $h$) and right child $z$ (children of height at most $h$ and $h$). Both have height $h+1$, so `w.height` is $h+2$.

RR and RL are the mirror images. In every case the subtree that replaces $z$ has height $h+2$, which is what `z.height` was before the insertion. So every ancestor of $z$ sees the same child heights as before, and its height and BF are unchanged. One single or double rotation fixes the whole tree.

## 1.2

Let $z$ be the lowest unbalanced node after a deletion from its left side, so $z$ is right-heavy and $y =$ `z.right`. Before the deletion, `z.left.height` was $h$, `y.height` was $h+1$ and `z.height` was $h+2$. After it, `z.left.height` is $h-1$ and the BF of $z$ is $-2$.

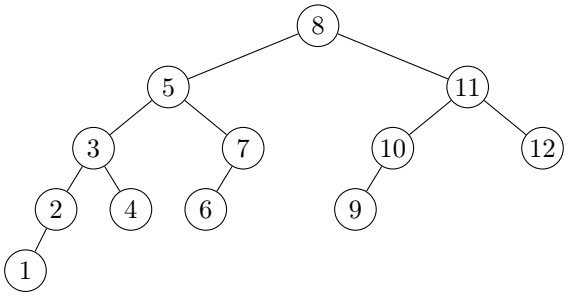
| Shape of $y$ | Rotation at $z$ | Height of repaired subtree |
|---|---|---|
| `y.right.height`$= h$, `y.left.height`$= h-1$ | RR (single) | $h+1$ |
| `y.left.height`$= h$, `y.right.height`$= h-1$ | RL (double) | $h+1$ |
| `y.left.height`=`y.right.height`$= h$ | RR (single) | $h+2$ |

Row 1: $z$ gets children of height $h-1$ and $h-1$, so `z.height` is $h$; $y$ gets children of height $h$ and $h$, so `y.height` is $h+1$. Row 2: with $w =$ `y.left`, the new root $w$ has left child $z$ and right child $y$, each of height $h$, so `w.height` is $h+1$. Row 3: $z$ gets children of height $h-1$ and $h$, so `z.height` is $h+1$; $y$ gets children of height $h+1$ and $h$, so `y.height` is $h+2$.

In rows 1 and 2 the repaired subtree is one shorter than it was before the deletion. Let $p$ be `z.parent`. If $z$ was the taller child of $p$, then $p$ becomes balanced and `p.height` drops by 1 (no rotation, but the shrinking continues upward). If $z$ was the shorter child of $p$, then $p$ now has BF 2 or $-2$ and needs its own rotation, which may shrink the subtree again. If the children of $p$ had equal heights, `p.height` is unchanged and the process stops. This repeats up the ancestors, possibly to the root. Row 3 cannot happen after an insertion (see 1.1), and it is the only case that stops the cascade.

### 1.3

The minimum node counts are N(3) = 1 + 2 + 1 = 4, N(4) = 1 + 4 + 2 = 7 and N(5) = 1 + 7 + 4 = 12. T5 is a root with T4 as `left` and T3 as `right`; T4 has T3 as `left` and T2 as `right`; T3 has T2 as `left` and T1 as `right`; T2 is a root with a single left child. Numbering the nodes in order gives:



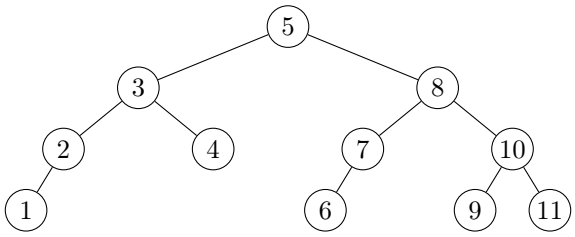

| key | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| height | 1 | 2 | 3 | 1 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 1 |
| BF | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |

Delete leaf **12**. Its parent 11 has BF 1 and 12 is on the shorter side, so 11 becomes unbalanced. Node 11 is the shorter child of the root 8 (BF 1), so once the subtree of 11 shrinks, 8 becomes unbalanced too. Both ancestors of 12 need a rotation.

### 1.4

**Step 1: node 11.** `11.left.height` is 2 and `11.right` is empty, so the BF of 11 is 2. The BF of 10 is 1, so this is the LL case: single right rotation at 11. The subtree becomes 10(9, 11) with height 2 (it was 3).

**Step 2: node 8.** `8.left.height` is 4 and `8.right.height` is now 2, so the BF of 8 is 2. The BF of 5 is 1, so this is the LL case again: single right rotation at 8. Node 5 becomes the root, 8 becomes `5.right`, and 7 (the old `5.right`) becomes `8.left`. Now `8.height` is 3 with BF 0, and `5.height` is 4 with BF 0.




| key | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| height | 1 | 2 | 3 | 1 | 4 | 1 | 2 | 3 | 1 | 2 | 1 |
| BF | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

Every BF is $-1$, 0 or 1, and the in-order traversal is 1 through 11, so this is a valid AVL tree.

### 2.1

With the size of an empty tree equal to 0:

`x.size = x.left.size + x.right.size + 1`

This uses only $x$ and its two children, so the augmentation theorem applies.

### 2.2

Let $y$ = `x.right`, $A$ = `x.left`, $B$ = `y.left` and $C$ = `y.right`. A left rotation at $x$ turns x(A, y(B, C)) into y(x(A, B), C). The updates, in this order:

`x.size = A.size + B.size + 1`

`y.size = x.size + C.size + 1`

$x$ must be updated first because the size of $y$ uses the new size of $x$. Both updates take $O(1)$.

### 2.3

Let $A$, $B$ and $C$ be perfect trees with $m$ nodes each (so $m = 2^k - 1$ for some $k$, and each has height $k$). Let $y$ have children $A$ and $B$, and let the root $x$ have children $y$ and $C$. Then $n = 3m + 2$, and this is a valid AVL tree: `y.left.height` equals `y.right.height` (BF 0), and `x.left.height` is $k + 1$ while `x.right.height` is $k$ (BF 1).

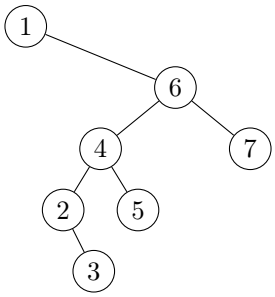
After a right rotation at $x$, the root is $y$ with children $A$ and x(B, C). Every node of $A$ moves up one level, so its depth decreases by 1. Every node of $C$ moves down one level, so its depth increases by 1. The nodes of $B$ keep their depth. So $2m$ stored depth values change in one rotation, which is about two thirds of all the nodes and at least proportional to $n$. A procedure that takes $O(1)$ time can write only a constant number of fields, so it cannot fix that many. Therefore `x.depth` cannot be maintained in $O(1)$ per rotation. Alternating a right rotation and a left rotation at the root repeats this every time, so $r$ rotations cost about $r \cdot n$ and amortizing does not help. The augmentation theorem does not apply because the depth of a node depends on its ancestors, not on the node and its children.

### 2.4

Search for the key starting at the root, adding 1 to a counter each time you move down a level. When you reach the node, the counter is its depth. This follows one root-to-node path, so it takes $O(h)$ time, and an AVL tree has $h = O(\log n)$. (With parent pointers you can instead climb from the node to the root and count the steps.)

### 3.1

Let $p$ denote the parent of $x$, and let $g$ denote the grandparent of $x$. Start with `x`=1, `p`=2, `g`=3. Rotate 2 over 3, then 1 over 2. Now 1 has right child 2, and 2 has right child 3, all hanging under 4. The next zig-zig (`p`=4, `g`=5) rotates 4 over 5, then 1 over 4. Now `1.right` is 4, `4.left` is 2 (with 3 below it), and `4.right` is 5. This continues until 1 is the root. For $n = 7$:




In general, 1 is the root, the even keys form one chain below it in decreasing order (starting from $n$ if $n$ is even, otherwise $n-1$), and each odd key $2j+1$ is `(2j).right`. The height is $\lfloor n/2 \rfloor + 2$ for $n \geq 3$ (checked by hand for $n = 3$ to 9), compared with $n$ before. So the height is about halved.

**Naive rotations.** Rotate 1 over 2, then over 3 (now `3.left` is 2), then over 4 (now `4.left` is 3), and so on. The result is root 1 with `1.right` equal to $n$, followed by the left chain $n, n-1, \ldots, 2$. The height is still $n$.

### 3.2

**Claim.** After naive `access(k)`, the root is $k$, the left subtree holds the keys 1 to $k-1$, and `k.right` is $n$ with the left chain $n, n-1, \ldots, k+1$ below it.

Base case ($k = 1$): this is the naive result in 3.1.

Inductive step: key $k+1$ is at the bottom of the left chain, which occupies depths 1 to $n-k$, so `(k+1).depth` is $n-k$. Rotating $k+1$ over its parent $k+2$, then over $k+3$, and so on up to $n$, re-hangs each earlier node as the left child of the next one, so the left chain from $n$ down to $k+2$ is preserved. When $k+1$ becomes `k.right`, one more rotation over $k$ makes $k$ (with its left subtree) the left child of $k+1$, and `(k+1).right` is still $n$. This is the claim for $k+1$.

Each access costs as many rotations as the depth of the key: $n-1$ for `access(1)`, and $n-k+1$ for `access(k)` when $k \geq 2$. The total is

(n - 1) + (1 + 2 + ... + (n - 1)) = (n - 1) + n(n - 1)/2 = (n - 1)(n + 2)/2,

which is on the order of $n^2$. Dividing by the $n$ operations gives an amortized cost on the order of $n$ per access.

### 3.3

Take a zig-zig where $g$ is at depth $d$: g(p(x(A, B), C), D). Sleator–Tarjan gives x(A, p(B, g(C, D))). The depths of the four subtrees change like this:

| subtree | A | B | C | D |
|---|---|---|---|---|
| depth before | $d+3$ | $d+3$ | $d+2$ | $d+1$ |
| depth after | $d+1$ | $d+2$ | $d+3$ | $d+3$ |
| change | $-2$ | $-1$ | $+1$ | $+2$ |

For the zig-zig x(A, B), p, g, the depth of A decreases by 2 and the depth of every other subtree stays about the same. The naive method also moves x up by 2, so the two methods give the same

tree and the same depths. The only difference between them is in the potential-function analysis, not in the shape of the tree. On a chain, neither method halves the depth.

Definitions: `x.size` is the number of nodes in the subtree of $x$, and `x.rank` = log2(`x.size`). The total potential is the sum of `rank` over all nodes. The amortized cost of a step is the number of rotations plus (new total potential minus old total potential). I write old and new for the values before and after the step.

A zig-zag step turns g(p(A, x(B, C)), D) into x(p(A, B), g(C, D)), using 2 rotations.

### 4.1

A node's rank depends only on which nodes are in its subtree. Only $x$, $p$ and $g$ get new subtrees. Nodes inside $A$, $B$, $C$ and $D$, and the ancestors of $g$, keep the same subtrees. So the change in total potential is

new rank($x$) + new rank($p$) + new rank($g$) - old rank($x$) - old rank($p$) - old rank($g$).

### 4.2

After the step, $x$ is at the position of $g$ and is the root of exactly the nodes that $g$ was the root of before. So new size($x$) = old size($g$), which means new rank($x$) = old rank($g$). Substituting this into the amortized cost (2 + change in potential), the terms new rank($x$) and old rank($g$) cancel:

amortized cost = 2 + new rank($p$) + new rank($g$) - old rank($x$) - old rank($p$).

### 4.3

Before the step, the subtree of $x$ is strictly inside the subtree of $p$, so size($x$) is less than size($p$), and so old rank($x$) $\leq$ old rank($p$). Therefore $-$old rank($p$) $\leq -$old rank($x$), and

amortized cost $\leq$ 2 + new rank($p$) + new rank($g$) - 2 $\cdot$ old rank($x$).

### 4.4

After the step, the subtree of $p$ is $p$, $A$ and $B$, and the subtree of $g$ is $g$, $C$ and $D$. These two sets do not overlap, and both are inside the subtree of $x$ (which also contains $x$ itself). So new size($p$) + new size($g$) $\leq$ new size($x$).

For any positive numbers $a$ and $b$, $a \cdot b \leq ((a+b)/2)^2$ (the average of two numbers is at least their geometric mean). Taking log2 of both sides gives log2($a$) + log2($b$) $\leq$ 2 $\cdot$ log2($a+b$) - 2. With $a$ = new size($p$) and $b$ = new size($g$), and since log2 is increasing,

new rank($p$) + new rank($g$) $\leq$ 2 $\cdot$ log2(new size($x$)) - 2 = 2 $\cdot$ new rank($x$) - 2.

Adding 2 to both sides: 2 + new rank($p$) + new rank($g$) $\leq$ 2 $\cdot$ new rank($x$).

### 4.5

Since new rank($x$) = old rank($g$) $\geq$ old rank($x$), the difference (new rank($x$) - old rank($x$)) is not negative, so 2 times it is at most 3 times it. Therefore the amortized cost of a zig-zag step is at

most $3 \cdot$ (new rank($x$) - old rank($x$)).

### 5.1

An internal node stores $M - 1$ keys (32 bytes each), $M - 1$ record pointers (8 bytes each) and $M$ child pointers (8 bytes each):

32(M - 1) + 8(M - 1) + 8M = 48M - 40 bytes, which must be at most 4096.

So $M$ is at most 4136 / 48 = 86.17, and the maximum order is $M = 86$. Check: $48 \cdot 86$ - 40 = 4088 bytes fits, but $M = 87$ needs 4136 bytes.

### 5.2

Every internal node other than the root has at least 86 / 2 = 43 children, so at least 42 keys. The root has at least 2 children and at least 1 key. For a tree of height $h$ (counting edges from the root to a leaf), the minimum number of keys is

$1 + (2 \cdot 42) + (2 \cdot 43 \cdot 42) + ... + (2 \cdot 43^{h-1} \cdot 42) = 1 + 2(43^h - 1) = 2 \cdot 43^h - 1$.

This must be at most 10,000,000. For $h = 4$: $2 \cdot 3{,}418{,}801 - 1 = 6{,}837{,}601$, which fits. For $h = 5$: $43^5 = 147{,}008{,}443$, which is far too large. So the maximum height is 4 edges, which is 5 levels of nodes.

### 5.3

A tree with height $h$ (counting nodes on the longest path) has at most $2^h$ - 1 nodes. Since $2^{23}$ = 8,388,608 is less than 10,000,001 and $2^{24}$ = 16,777,216 is at least 10,000,001, the minimum height is 24.

The minimum number of nodes in an AVL tree of height $h$ is N(h) = Fib(h + 2) - 1, where Fib(1) = Fib(2) = 1, Fib(3) = 2, and so on (check: N(5) = Fib(7) - 1 = 13 - 1 = 12). Fib(35) = 9,227,465 gives N(33) = 9,227,464, which is at most 10,000,000. Fib(36) = 14,930,352 gives N(34) = 14,930,351, which is more. So the maximum height is 33.

### 5.4

The root is cached, so every other node on the search path costs one disk read.

| | levels | disk reads | time at 10 ms per read |
|---|---|---|---|
| B-tree, minimally filled | 5 | 4 | 40 ms |
| AVL tree, worst case | 33 | 32 | 320 ms |
| AVL tree, best case | 24 | 23 | 230 ms |

On disk, the cost is the number of blocks read, not the number of comparisons. A B-tree node fills a whole block and holds up to 85 keys, so each read narrows the search by a factor of 43 to 86, while a binary tree node narrows it by a factor of 2. The B-tree height is about log base $M$ of $N$ instead of log2 of $N$, which means about 5 to 6 times fewer reads. The search inside a block takes microseconds in memory, which is negligible.

### 6.1

```
Method: prefix_sum (x, k, inclusive)
Input: A node x of an augmented BST, a number k, and a flag inclusive

  if x == nil
    return 0
  if (x.key < k) or (inclusive and x.key == k)
    return x.left.sum + x.key + prefix_sum(x.right, k, inclusive)
  else
    return prefix_sum(x.left, k, inclusive)

Output: The sum of all keys <= k (if inclusive) or < k (otherwise)

Method: range_sum (T, low, high)
Input: An augmented BST root T where each node x has fields x.key,
       x.left, x.right, x.sum, and interval [low, high] with low <= high

  return prefix_sum(T, high, true) - prefix_sum(T, low, false)

Output: The exact sum of all keys in T satisfying low <= key <= high
```

The sum field is maintained with `x.sum = x.left.sum + x.right.sum + x.key`.

### 6.2

**Loop invariant.** At the start of each iteration, $s$ plus the sum of the qualifying keys in the subtree of $x$ equals the sum of the qualifying keys in the whole tree. (A key qualifies if it is at most $k$ for the inclusive call, or less than $k$ for the other.)

- At the start, $x$ is the root and $s$ is 0.
- If `x.key` qualifies: every key in `x.left` is at most `x.key`, so all of them qualify and add up to `x.left.sum`. The key `x.key` qualifies too. The only keys left to examine are in `x.right`, which is where the loop goes. So adding `x.left.sum + x.key` to $s$ keeps the invariant.
- If `x.key` does not qualify: every key in `x.right` is at least `x.key`, so none of them qualify either. Nothing is added and the loop goes to `x.left`.
- When $x$ is empty, no keys are left, so $s$ is the sum of all qualifying keys.

So the first call returns the sum of all keys $\leq$ high and the second returns the sum of all keys $<$ low. Because low $\leq$ high, every key below low is also at most high. Subtracting removes exactly those keys, and leaves exactly the keys with low $\leq$ key $\leq$ high, each counted once.

### 6.3

At each node, prefixSum looks at both the left subtree and the right subtree: it adds x.left.sum and then goes on to the other side. So each call does constant work plus work on two subtrees of height

$h-1$:

$T(h) = 2T(h-1) + c, with T(0) = c$

Expanding to the root node gives$T(h) = c(2^(h+1) - 1)$, which is $O(2^h) = O(n)$. Since the tree is balanced, $h = O(logn)$, so the algorithm runs in $O(logn)$ time.

In an AVL tree of height $h$ with $n$ nodes, $n$ is at least N(h) = Fib(h + 2) - 1. Since Fib(m) is at least $1.618^{m-2}$ (true for $m = 1$ and $m = 2$, and each later Fibonacci number is the sum of the previous two), $n$ is at least $1.618^h$ - 1. Taking log base 1.618 gives $h \leq$ log base 1.618 of $(n + 1)$, which is about $1.44 \cdot \log 2(n+1)$. So $h$ is $O(\log n)$, and `range_sum` runs in $O(\log n)$ worst-case time, no matter how many keys are in the range.